

TEAM OVERCLOCK

Product Requirements Document

TARA

Trust And Relationship Analysis

QoreID proves who they are. TARA reveals who they are connected to. A network-based identity trust layer built live on a fresh clone of the VERA repository — same graph architecture, new domain: coordinated identity fraud invisible to individual verification.

Event	TiT 6.0 — Thrive-In-Tech Hackathon, Lagos
Date	August 21, 2026 — Golfview Suites, Ikeja GRA
Track	Digital Identity & Trust (QoreID-sponsored)
Base Repo	Fresh clone of the VERA/GRACE repository
Build Window	09:30 AM start — 04:00 PM hard submission deadline
Team	Team Overclock
Audience	Every team member, Claude Code / Codex during the build

Context, Rules & The Honest Framing

READ THIS BEFORE ANYTHING ELSE

TiT 6.0's published Hackathon Guide states: "Build during the event. All code and design must be created on the day. Pre-built codebases or existing products are not permitted." Team Overclock has separate written permission via email from the organisers to build from the existing VERA codebase for this specific submission, since VERA has not been pitched or used at any prior hackathon that overlaps with this one.

Keep that email accessible on the day. Be ready to state clearly, specifically, and without hedging in Q&A; exactly what was built fresh today versus what was adapted from the existing repo. Vagueness here is the single biggest risk to credibility with the judging panel, not the reuse itself.

The Repositioning

VERA already means Verification Engine for Risk Analysis. Today it verifies financial transactions for AML compliance. For TiT 6.0, the same graph architecture is redirected at a different question: not "is this transaction suspicious" but "is this verified identity secretly connected to other verified identities in a way that forms a fraud ring."

QoreID answers whether a person is real. TARA answers whether they are acting alone. Every identity in the demo passes QoreID verification individually. Together, a subset of them form a coordinated ring — shared device, shared address, shared onboarding window. That gap between individual verification and network reality is the entire product.

What Changes From VERA — Be Precise About This

Dimension	Current VERA	TARA (TiT 6.0)
Data source	Squad payment webhooks	QoreID verification API responses
Graph nodes	Financial entities	Verified human identities
Graph edges	Transactions between entities	Shared attributes (device, address, phone window, employer)
Detection patterns	POS rings, shell webs, layered chains	Shared-attribute clusters, identity fragmentation, coordinated onboarding velocity
Output artifact	NFIU Suspicious Transaction Report	Identity Trust Verdict (Approve / Review / Reject)
End user	Bank compliance officer	Any platform onboarding users — lenders, fintechs, marketplaces
Action layer	Squad quarantine transfer	Onboarding decision gate

What Is Reused Without Modification

- FastAPI backend skeleton — routing, config, DB session handling
- Graph data structure and traversal logic (in-memory, no Neo4j provisioning needed on the day)
- D3.js force-directed graph explorer on the frontend — this is the single biggest time-saver
- RapidFuzz-based fuzzy matching for identity fragmentation detection
- Risk score propagation math — direct score plus neighbour decay
- React component shell, Tailwind design tokens, TanStack Query setup

What Is Built Fresh On The Day

- QoreID API adapter — the single most important fresh build, must be genuinely live
- Identity attribute graph model — new node and edge schema
- Three identity-specific detection patterns (Section 04)
- Verdict gate logic and UI (Approve / Review / Reject)
- Explanation panel wording and evidence formatting
- All copy, naming, and branding — VERA to TARA across the app

02

Repository Setup — First 20 Minutes

Do this before the 9:20 AM QoreID track brief if possible, so the team is building the moment the brief ends. Do not wait for the brief to start cloning.

Step 1 — Clone and branch

Terminal

```
git clone https://github.com/Techdee1/GRACE tara
cd tara
git checkout -b tara-tit6
git remote rename origin upstream
git remote add origin <new-empty-repo-url-for-submission>
```

Create a fresh, empty public GitHub repo for the TARA submission before the day. Push to that origin, not to the original VERA repo — the submission rules require a public repo containing the full source, and judges will look at commit history. A single-day commit history on a fresh repo is exactly the story you want them to see.

Step 2 — Global rename pass (do this early, not late)

Terminal — case-sensitive find and replace across the repo

```
# Backend
grep -rl "VERA" backend/app --include=*.py | xargs sed -i 's/VERA/TARA/g'
grep -rl "vera" backend/app --include=*.py | xargs sed -i 's/vera/tara/g'

# Frontend
grep -rl "VERA" src --include=*.jsx,tsx,ts,js | xargs sed -i 's/VERA/TARA/g'
grep -rl "vera" src --include=*.jsx,tsx,ts,js | xargs sed -i 's/vera/tara/g'

# Update package.json, README, index.html title
sed -i 's/VERA/TARA/g; s/vera/tara/g' package.json README.md index.html
```

WHY RENAME EARLY, NOT LATE

Renaming at 3:45 PM under deadline pressure is how a stray "VERA" ends up on screen during a live demo in front of a QoreID judge. Do the mechanical rename in the first 20 minutes, then build TARA-native from that point forward. New code you write after this point should never mention VERA at all.

Step 3 — Strip what you will not use today

Comment out or remove references to Squad webhooks, Neo4j connection setup, Celery/Redis task queues, and STR generation. None of these apply to TARA's scope. Removing them prevents confused judges asking why a payments webhook exists in an identity trust product.

backend/app/main.py — comment out on startup

```
# from app.api.routes import webhooks # Squad — not used in TARA
# from app.services.str_service import STRService # NFIU STR — not used in TARA
# app.include_router(webhooks.router) # disabled for TiT 6.0 build
```

Resulting folder structure

```
tara/
  ■■■ backend/
  ■ ■■■ app/
  ■ ■ ■■■ api/routes/
  ■ ■ ■ ■■■ identities.py # NEW – replaces transactions.py
  ■ ■ ■ ■■■ graph.py # REUSED – minor field changes
  ■ ■ ■ ■■■ patterns.py # NEW – 3 identity detection patterns
  ■ ■ ■ ■■■ verdict.py # NEW – replaces str_drafts.py
  ■ ■ ■■■ services/
  ■ ■ ■ ■■■ qoreid_service.py # NEW – the critical adapter
  ■ ■ ■ ■■■ graph_service.py # REUSED – node/edge model updated
  ■ ■ ■ ■■■ pattern_service.py # NEW – pattern detection logic
  ■ ■ ■ ■■■ entity_resolution.py # REUSED – RapidFuzz, untouched
  ■ ■ ■■■ models/
  ■ ■ ■■■ identity.py # NEW – replaces entity.py
  ■ ■ ■■■ relationship.py # NEW – replaces transaction.py
  ■■■ src/
  ■ ■■■ pages/
  ■ ■ ■■■ Dashboard.jsx # REUSED – copy and stat cards updated
  ■ ■ ■■■ GraphExplorer.jsx # REUSED – near-identical, relabeled
  ■ ■ ■■■ IdentityDetail.jsx # NEW – replaces EntityDetail.jsx
  ■ ■ ■■■ VerdictDetail.jsx # NEW – replaces STRDetail.jsx
  ■ ■■■ components/
  ■ ■■■ TrustScoreCard.jsx # REUSED – renamed, logic unchanged
  ■■■ data/
  ■■■ demo_identities.json # NEW – 24 synthetic demo identities
```

03

QoreID Integration — Build This First

THIS IS THE SINGLE HIGHEST-RISK, HIGHEST-VALUE COMPONENT

A QoreID engineer sits on the judging panel. If the integration is faked or flaky when it matters, the credibility of the entire pitch collapses in front of the one judge most qualified to notice. Build this before anything else, while the team is fresh, and confirm it end-to-end before 11:00 AM.

Immediate actions at 9:20 AM

- Attend the QoreID track brief in full — do not multitask during it
- Get sandbox API credentials from QoreID's on-site team or briefing materials
- Confirm the exact verification endpoints available in sandbox (BVN, NIN, or both)
- Ask directly: what does QoreID want to see teams build on top of their API today
- Note any QoreID-specific judging hints given during the brief — these often matter more than the general rubric for the track prize

QoreID Service Adapter

backend/app/services/qoreid_service.py (NEW FILE)

```
import httpx
from app.core.config import settings

QOREID_BASE_URL = "https://api.qoreid.com/v1" # confirm exact URL at track brief

class QoreIDService:
    def __init__(self):
        self.client = httpx.AsyncClient(
            base_url=QOREID_BASE_URL,
            headers={
                "Authorization": f"Bearer {settings.qoreid_api_key}",
                "Content-Type": "application/json",
            },
            timeout=15.0,
        )

    async def verify_identity(self, bvn: str = None, nin: str = None,
                             first_name: str = None, last_name: str = None):
        """
        Calls QoreID's verification endpoint.
        Confirm exact payload shape against their docs at the track brief —
        this is a reasonable placeholder structure to adapt from.
        """
        payload = {
            "idNumber": bvn or nin,
            "idType": "bvn" if bvn else "nin",
            "firstname": first_name,
            "lastname": last_name,
        }
        response = await self.client.post("/identities/verify", json=payload)
        response.raise_for_status()
        data = response.json()

        return {
```

```
"verified": data.get("status") == "verified",
"confidence": data.get("confidence", None),
"matched_name": data.get("data", {}).get("fullName"),
"matched_dob": data.get("data", {}).get("dateOfBirth"),
"matched_phone": data.get("data", {}).get("phoneNumber"),
"raw_response": data, # keep for the on-screen demo proof
}
```

Endpoint that wraps the adapter

backend/app/api/routes/identities.py (NEW FILE)

```
from fastapi import APIRouter, HTTPException
from app.services.qoreid_service import QoreIDService
from app.services.graph_service import upsert_identity_node

router = APIRouter(prefix="/identities", tags=["identities"])
qoreid = QoreIDService()

@router.post("/verify")
async def verify_and_ingest(payload: dict):
    try:
        result = await qoreid.verify_identity(
            bvn=payload.get("bvn"),
            first_name=payload.get("first_name"),
            last_name=payload.get("last_name"),
        )
    except Exception as e:
        raise HTTPException(status_code=502, detail=f"QoreID call failed: {e}")

    if not result["verified"]:
        return {"status": "rejected", "reason": "identity_not_verified"}

    node = upsert_identity_node(
        name=result["matched_name"],
        device_id=payload.get("device_id"),
        address=payload.get("address"),
        phone=result["matched_phone"],
        employer=payload.get("employer"),
        verified_at="now",
    )

    return {
        "status": "verified",
        "identity_id": node.id,
        "qoreid_raw": result["raw_response"], # show this on screen live
    }
```

DEMO CREDIBILITY TIP

During the pitch, do not just show a green checkmark. Show the raw QoreID JSON response briefly on screen, even for two seconds. That single visual beat does more to establish authenticity with a technical judge than any amount of narration.

04

Detection Patterns — Build Exactly Three

Do not attempt more than three patterns. Each one below is scoped to be achievable in under 90 minutes given the reused graph infrastructure. Build Pattern 1 completely before starting Pattern 2.

Pattern 1 — Shared Attribute Cluster (P0, build first)

Multiple verified identities sharing a device fingerprint, physical address, or employer with no CAC registration. The clearest, most visual pattern — build this one to be flawless, since it will very likely be what the graph shows during the live pitch.

`backend/app/services/pattern_service.py — Pattern 1`

```
def detect_shared_attribute_clusters(graph: dict, min_cluster_size: int = 4):
    """
    Groups identities by shared device_id, address, or employer.
    Flags any group at or above min_cluster_size.
    """
    clusters = {}
    for identity_id, node in graph.items():
        for attr_key in ["device_id", "address", "employer"]:
            attr_val = node.get(attr_key)
            if not attr_val:
                continue
            cluster_key = f"{attr_key}:{attr_val}"
            clusters.setdefault(cluster_key, []).append(identity_id)

    findings = []
    for cluster_key, members in clusters.items():
        if len(members) >= min_cluster_size:
            attr_type, attr_val = cluster_key.split(":", 1)
            findings.append({
                "pattern_type": "shared_attribute_cluster",
                "shared_attribute": attr_type,
                "value": attr_val,
                "identity_ids": members,
                "evidence_summary":
                    f"{len(members)} verified identities share the same "
                    f"{attr_type.replace('_', ' ')}: {attr_val}",
            })
    return findings
```

Pattern 2 — Identity Fragmentation (P0)

The same human operating multiple verified identities via slight name variations. Reuses your existing RapidFuzz entity resolution almost unchanged — this is the fastest pattern to build given what already exists in the repo.

`backend/app/services/pattern_service.py — Pattern 2`

```
from rapidfuzz import fuzz

def detect_identity_fragmentation(graph: dict, similarity_threshold: int = 85):
    """
    Reuses existing fuzzy name matching logic from entity_resolution.py.
    Flags identity pairs whose names are highly similar but registered
    as separate verified identities.
```

```

"""
findings = []
identities = list(graph.values())
for i in range(len(identities)):
    for j in range(i + 1, len(identities)):
        a, b = identities[i], identities[j]
        score = fuzz.token_sort_ratio(a["name"], b["name"])
        if score >= similarity_threshold and a["id"] != b["id"]:
            # Extra signal: same phone area code or overlapping address
            same_context = (
                a.get("address") == b.get("address")
                or (a.get("phone", "")[:4] == b.get("phone", "")[:4])
            )
            if same_context:
                findings.append({
                    "pattern_type": "identity_fragmentation",
                    "identity_ids": [a["id"], b["id"]],
                    "similarity_score": score,
                    "evidence_summary":
                        f"'{a['name']}' and '{b['name']}' are "
                        f"{score}% name-similar and share context – "
                        f"likely the same person under two verified identities",
                })
return findings

```

Pattern 3 — Coordinated Onboarding Velocity (P1)

N identities completing QoreID verification within a tight time window, sharing a device or referral source. Only build this after Patterns 1 and 2 are working end-to-end. Classic mule-farm signature.

[backend/app/services/pattern_service.py — Pattern 3](#)

```

from datetime import datetime, timedelta

def detect_coordinated_onboarding(graph: dict, window_hours: int = 48,
min_group_size: int = 4):
    """
    Groups identities verified within the same rolling time window
    that also share a device_id or referral_source.
    """
    findings = []
    by_device = {}
    for node in graph.values():
        key = node.get("device_id") or node.get("referral_source")
        if not key:
            continue
        by_device.setdefault(key, []).append(node)

    for key, members in by_device.items():
        if len(members) < min_group_size:
            continue
        times = sorted(m["verified_at"] for m in members)
        earliest, latest = times[0], times[-1]
        if (latest - earliest) <= timedelta(hours=window_hours):
            findings.append({
                "pattern_type": "coordinated_onboarding",
                "identity_ids": [m["id"] for m in members],
                "window_hours": window_hours,
                "evidence_summary":
                    f"{len(members)} identities verified within "
                    f"{window_hours} hours, sharing device or referral source",
            })

```

```
})  
return findings
```

05

Trust Score & Verdict Gate

The scoring math is reused almost entirely from VERA's existing risk propagation logic. Only the labels and thresholds change.

`backend/app/services/pattern_service.py — scoring`

```
def compute_trust_score(identity_id: str, findings: list, graph: dict) -> dict:
    """
    Direct involvement in a flagged pattern sets the base score.
    Connected identities inherit a decayed score — reused from VERA.
    """
    direct_findings = [f for f in findings if identity_id in f["identity_ids"]]

    if not direct_findings:
        return {"score": 0.05, "verdict": "APPROVE", "evidence": []}

    base_score = min(0.95, 0.35 + 0.2 * len(direct_findings))

    if base_score >= 0.7:
        verdict = "REJECT_REVIEW" # never auto-reject — always human review
    elif base_score >= 0.4:
        verdict = "REVIEW"
    else:
        verdict = "APPROVE"

    return {
        "score": round(base_score, 2),
        "verdict": verdict,
        "evidence": [f["evidence_summary"] for f in direct_findings],
    }
```

NEVER AUTO-REJECT

The verdict gate has three states, not two. High-confidence findings map to REJECT_REVIEW, not REJECT. A human always makes the final call. This is both the responsible-AI answer to an obvious judge question and simply correct — a shared address between two flatmates is not fraud, and the system should never claim otherwise with full automation.

Verdict Gate Endpoint

`backend/app/api/routes/verdict.py (NEW FILE)`

```
from fastapi import APIRouter
from app.services.pattern_service import (
    detect_shared_attribute_clusters,
    detect_identity_fragmentation,
    detect_coordinated_onboarding,
    compute_trust_score,
)
from app.services.graph_service import get_full_graph

router = APIRouter(prefix="/verdict", tags=["verdict"])

@router.get("/{identity_id}")
async def get_verdict(identity_id: str):
    graph = get_full_graph()
```

```
findings = (
detect_shared_attribute_clusters(graph)
+ detect_identity_fragmentation(graph)
+ detect_coordinated_onboarding(graph)
)

result = compute_trust_score(identity_id, findings, graph)

return {
"identity_id": identity_id,
"trust_score": result["score"],
"verdict": result["verdict"],
"evidence": result["evidence"],
"explanation": build_explanation(result["evidence"]),
}

def build_explanation(evidence: list) -> str:
if not evidence:
return "No relationship signals found. This identity appears independent."
return "Flagged because: " + " Also: ".join(evidence)
```

06

Demo Dataset — The Loan Stacking Ring

24 synthetic identities. Eight form the fraud ring, the remaining sixteen are clean, independent identities that exist purely so the graph does not look artificially small or obviously rigged. Build this file early — it unblocks frontend work in parallel with backend pattern development.

`data/demo_identities.json — the ring (8 of 24 identities)`

```
[
  {
    "name": "Adaeze Nwankwo", "bvn": "22198877001",
    "device_id": "DEV-9F31A", "address": "14 Allen Ave, Ikeja",
    "phone": "08031234501", "employer": "Zenta Logistics Ltd",
    "verified_at": "2026-08-21T06:12:00"
  },
  {
    "name": "Adaeze N. Nwankwo", "bvn": "22198877002",
    "device_id": "DEV-9F31A", "address": "14 Allen Ave, Ikeja",
    "phone": "08031234502", "employer": "Zenta Logistics Ltd",
    "verified_at": "2026-08-21T06:40:00"
  },
  {
    "name": "Chidinma Okoro", "bvn": "22198877003",
    "device_id": "DEV-9F31A", "address": "14 Allen Ave, Ikeja",
    "phone": "08031234503", "employer": "Zenta Logistics Ltd",
    "verified_at": "2026-08-21T07:05:00"
  },
  {
    "name": "Ifeoma Balogun", "bvn": "22198877004",
    "device_id": "DEV-7C112", "address": "14 Allen Ave, Ikeja",
    "phone": "08031234504", "employer": "Zenta Logistics Ltd",
    "verified_at": "2026-08-21T07:38:00"
  }
]
// ... 4 more ring members, same pattern, one added variable each
// ... plus 16 clean, unconnected identities for graph realism
```

WHY 24 AND NOT FEWER

A graph with only 8 nodes and no noise looks staged. 24 nodes with a visible cluster of 8 lighting up red against a field of green looks like a real detection system finding a needle, not a system built to only ever find the needle you planted. This single detail materially improves how credible the demo feels to a judge who has seen a dozen hackathon graphs already today.

07

Frontend Adaptation

Reused near-untouched

- GraphExplorer.jsx — same D3 force-directed layout, node labels now show identity names instead of entity IDs, colour scale unchanged (red/amber/green)
- Dashboard.jsx layout shell — metric cards restyled with new labels
- TrustScoreCard.jsx — logic identical, badge text changes from risk terms to verdict terms (VERIFIED / REVIEW / HIGH RISK)

New screens to build

Screen	Replaces	Core content
IdentityDetail.jsx	EntityDetail.jsx	QoreID verification badge, relationship signals list, connected identities
VerdictDetail.jsx	STRDetail.jsx	Trust score, verdict badge, evidence list, explanation panel, Approve/Review/Reject actions
VerifyIdentity.jsx	new	Simple form — submit an identity live during the demo, triggers the QoreID call on screen

Dashboard copy changes

src/pages/Dashboard.jsx — label updates

```
// Before (VERA)
"Entities Monitored" -> "Identities Verified"
"Open Alerts" -> "Flagged Clusters"
"High Risk Entities" -> "Identities Under Review"
"STR Drafts" -> "Verdicts Pending"
```

08

Build Schedule — Hour By Hour

Time	Task	Priority
09:00 – 09:20	General opening brief	—
09:20 – 09:40	QoreID track brief — attend fully, get credentials	—
09:40 – 10:00	Repo clone, branch, global VERA→TARA rename pass	P0
10:00 – 11:00	QoreID adapter built and confirmed working end-to-end	P0
11:00 – 12:00	Identity graph model, demo dataset (24 identities), Pattern 1	P0
12:00 – 12:30	Pattern 2 (identity fragmentation)	P0
12:30 – 13:00	Lunch break	—
13:00 – 13:20	Mid-point mentor round — get direction check	—
13:20 – 14:00	Verdict gate, trust score, explanation panel	P0
14:00 – 14:45	Frontend: IdentityDetail, VerdictDetail, dashboard copy	P0
14:45 – 15:15	Pattern 3 (coordinated onboarding) — only if on schedule	P1
15:15 – 15:45	Full demo run-through, fix breakage, polish graph visual	P0
15:45 – 16:00	Freeze code, make GitHub repo public, final commit	P0
16:00	HARD DEADLINE — submission locked, no extensions	—
16:00 – 16:30	Finalise pitch deck + 200-300 word written summary	P0
16:30 – 17:30	Rehearse pitch — at least twice, out loud, timed	P0
17:30	Team pitch presentations begin	—

CUT LIST IF BEHIND SCHEDULE AT 14:00

Drop Pattern 3 entirely. Drop any LLM narrative generation. Drop the audit log view. Drop styling polish beyond "looks intentional." Two patterns working flawlessly beats three patterns half-working. The judging guide states this explicitly: a working simple solution always beats an ambitious broken one.

Submission Requirements Checklist

All four items are required by 4:00 PM WAT. A submission is incomplete, and therefore unjudged, if any one is missing.

Item	Requirement	Owner
Working Prototype	Live product or recorded walkthrough — judges must see it in action during the pitch	Whole team
GitHub Repository	Public repo, full source code, set to public before deadline	Backend lead
Pitch Deck	5–10 slides: problem, solution, demo, impact, tech stack, team. PDF or Google Slides link	Pitch lead
Written Summary	200–300 words: problem, approach, impact. Via organiser form	Pitch lead

Judging Criteria — Build Against Every One Explicitly

Criterion	Weight	How TARA addresses it
Innovation	25%	Network-layer identity trust on top of standard verification — a real gap neither QoreID nor RegFyl-style tools currently close
Technical Implementation	25%	Live QoreID API calls, working graph detection across 3 patterns, functional verdict gate — not a mockup
Impact & Relevance	20%	Loan stacking and mule identity farms are documented, active Nigerian fraud patterns with real financial cost
Design & UX	15%	D3 graph visualisation makes the fraud ring immediately, visually obvious — no explanation needed
Pitch & Presentation	15%	Rehearsed live QoreID demo, clear hook, honest Q&A; about what was built today

10

Five Minute Demo Script

0:00 – 0:30 — The Hook

"These eight people all passed identity verification. Every one of them is a real, verifiable Nigerian. Watch what happens when we stop looking at them one at a time."

0:30 – 1:30 — Live QoreID Verification

Submit a new identity live through the VerifyIdentity form. Show the actual QoreID API response on screen, even briefly. "This is QoreID, live, right now." The identity enters the graph.

1:30 – 3:00 — The Reveal

Open the Graph Explorer. The eight-identity ring lights up red against sixteen green, independent nodes. Walk through the shared attributes out loud — same device, same address, verified within a 48-hour window.

3:00 – 4:00 — The Why And The Verdict

Open the explanation panel. Read the concrete evidence aloud. Show the Trust Score. State the verdict clearly: "Review — not Reject. A human makes the final call, always."

4:00 – 5:00 — The Close

"Identity verification tells you someone is real. It does not tell you they are acting alone. TARA is the layer that answers the second question. Built today, live, on QoreID's own API."

Q&A; Preparation

"How is this different from what QoreID already does?"

We do not compete with QoreID, we depend on it. Verification is the input. Without a trusted identity signal there is no graph worth building. We add the relational layer on top.

"A family sharing an address is not fraud. How do you handle false positives?"

Thresholds are configurable. A cluster of two or three is normal and does not fire. Nothing is auto-rejected — the output is Review, and a human decides with the full evidence in front of them.

"You are building a graph of people's identities. What about privacy?"

Deploys inside the customer's own infrastructure. Data never leaves their environment. Purpose-limited to fraud prevention, NDPA-aligned by design.

"What did you actually build today, specifically?"

Answer this directly. Name the QoreID adapter, the identity attribute graph model, the three detection patterns, the verdict gate. Do not be vague.

THE ONE LINE TO REMEMBER

"QoreID proves who they are. TARA reveals who they are connected to." Every answer in Q&A; should trace back to this sentence.